

MSAGLJS: Graph Layout, Edge Routing, and Tiling for the Web

Anonymous author(s)

Anonymous affiliation(s)

Abstract

We present MSAGLJS, an open-source TypeScript library for graph visualization in web browsers, distributed as a set of NPM packages¹. It provides a set of layout algorithms, a set of edge routing algorithms, and two renderers that run in an Internet browser: one based on SVG and another based on WebGL via the deck.gl framework. In this paper we concentrate on two features of the latter renderer.

First, a *tiling scheme* that turns a large graph into a navigable map with *semantic zoom*: at every zoom level the labels of the highest-ranked nodes remain readable, just as the names of major geographical features stay readable on digital maps—making the browsing experience familiar to anyone used to online maps. Second, a *corridor routing* method that routes each edge along the shortest path in homotopy class around the node obstacles, based on the Constrained Delaunay Triangulation. The whole pipeline runs client-side, without a dedicated server, on a phone or a laptop. In this configuration the WebGL renderer interacts smoothly with graphs of up to 10 000 nodes and 40 000 edges. **Re-run benchmarks to confirm the maximum graph size; current 10 000 / 40 000 figures are placeholders.**

MSAGLJS is available at <https://github.com/microsoft/msagljs>.

2012 ACM Subject Classification Human-centered computing → Graph drawings; Theory of computation → Computational geometry

Keywords and phrases Graph visualization, edge routing, Constrained Delaunay Triangulation, tiling, WebGL

Category Track 2, Long Paper

Generative AI Declaration Generative AI was used for editorial assistance in the preparation of this article.

1 Introduction

Visualizing graphs in web browsers without a dedicated server presents unique challenges: the runtime is single-threaded, the per-tab JavaScript heap is capped at roughly 4 GB,² and users expect the smooth pan-and-zoom experience familiar from online maps. MSAGLJS³ is an open-source TypeScript library that addresses these challenges by combining efficient layout algorithms, a novel edge routing method, and a tiling scheme for level-of-detail rendering.

MSAGLJS is consumed as a set of NPM packages and renders graphs using WebGL through the deck.gl framework [4]. It supports the layered Sugiyama scheme [42], Pivot MDS [16], and IPSep-CoLa [22], with edges routed around node obstacles. Throughout this paper our typical example is a social network laid out by IPSep-CoLa; the methods we describe, however, are agnostic to the layout algorithm and apply to any node placement in which the nodes do not overlap. The rendering pipeline builds a hierarchy of tiles—analogue

¹ @msagl/core, @msagl/drawing, @msagl/parser, @msagl/renderer-webgl — all available on <https://www.npmjs.com/>.

² In current Chrome, V8 with pointer compression caps the per-tab JavaScript heap at approximately 4 GB (2^{32} bytes).

³ <https://github.com/microsoft/msagljs>



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

34th International Symposium on Graph Drawing and Network Visualization (GD 2026).

Editors: TBD; Article No. 0; pp. 0:1–0:14

Leibniz International Proceedings in Informatics



LIPIC Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

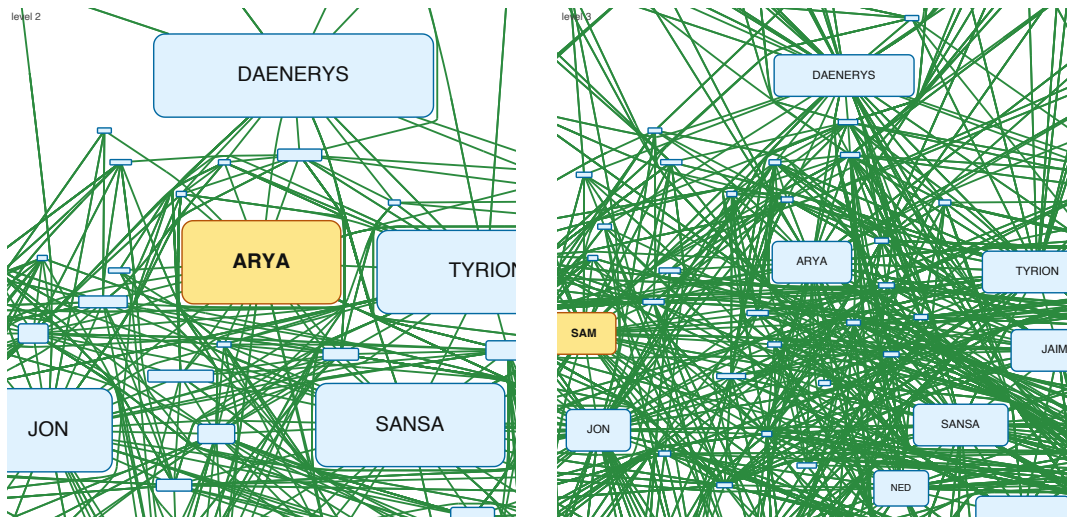


Figure 1 Overview of the per-level-graph tiling scheme presented in Section 4, on the Game of Thrones graph. Two adjacent pyramid levels are shown, cropped to the same window; the coarser level is on the left, the finer one on the right. For each level a new graph is assembled from a PageRank-ranked subset of the nodes, and the top-ranked landmark, highlighted in amber, enters that level enlarged by a factor of two in each dimension. Remaining survivors, shown in light blue, are kept at a node-specific scale $s_v \in [0.25, 2]$ that shrinks with rank so that they do not overlap. Edges are rerouted per level with a sleeve-hint A* built from the finer-level route, which keeps the coarse shape close to the fine one and yields smooth transitions as the user pans and zooms.

to map tiles—so that at any zoom level only a bounded number of entities are drawn. Tiles are delivered to the browser through the standard `TileLayer` of the `@deck.gl/geo-layers` package, giving smooth pan and zoom. Live demos are available online: a WebGL renderer for large graphs⁴ and an SVG renderer for smaller, editable graphs⁵.

This paper makes two main contributions:

1. A *tiling scheme* for large graph visualization that builds a hierarchical tile pyramid, limits visible entities per tile using PageRank-based filtering, simplifies edge routes at coarser levels, and integrates with the WebGL renderer for real-time pan and zoom.
2. A *corridor routing* algorithm that routes edges on the dual graph of a Constrained Delaunay Triangulation, using per-source batched Dijkstra trees followed by the classical funnel algorithm to produce shortest-path geodesics in homotopy classes.

2 Related Work

2.0.0.1 Graph visualization tools.

Graphviz [26, 6] is a long-standing graph drawing tool supporting multiple layout algorithms, including the Sugiyama method and Scalable Force-Directed Placement [9]; it does not support tiling or WebGL rendering. Our library builds on the earlier MSAGL desktop layout engine [38] that did not have the features described here. On the web, Sigma.js [10] and Cytoscape.js [24] provide interactive graph rendering, but rely on straight-line or generic

⁴ <https://microsoft.github.io/msagljs/renderer-webgl/index.html>

⁵ <https://microsoft.github.io/msagljs/renderer-svg/index.html>

spline edges without obstacle-avoiding routing and without a pyramid of precomputed tiles. ReGraph [8] uses WebGL to render large graphs with straight-line edges and supports interactive cluster expansion but not tiling. Cosmograph [3] uses GPU-accelerated force-directed layout for million-node graphs with straight-line edges, without tiling. GraphMaps [37] supports tiling and level-of-detail for large graphs but runs only on Windows and does not optimize edge routes. Cornac [40] handles huge graphs with tiles and edge bundling [35, 33] but requires a multi-server backend. Earlier navigation schemes for large graphs include ASK-GraphView [14], which organizes nodes into a hierarchical cluster tree, and topological fisheye views [25], which distort a multilevel layout [28, 34] under the focus; neither targets browser-side tile pyramids or per-level edge rerouting. Hierarchical edge bundling [32] and force-directed bundling [33] are complementary clutter-reduction techniques applied *after* edges have been routed.

2.0.0.2 Shortest paths and edge routing.

Our corridor algorithm walks the dual of a Constrained Delaunay Triangulation [41] and relies on the classical funnel algorithm for extracting a geodesic from a triangle strip [36, 17, 27, 5]. The optimal algorithm for Euclidean shortest paths among polygonal obstacles [31] and the recent $O(m \log^{2/3} n)$ single-source shortest-path algorithm that breaks the sorting barrier [21] are remarkable theoretical results, but they are not practical: both build elaborate global data structures that we avoid. Graphviz builds the full visibility graph for edge routing in force-directed layouts, which has $O(n^2)$ edges. For Sugiyama layouts, it uses the funnel algorithm [19] but requires a simple containing polygon. yWorks [13] offers “Organic edge routing” based on force-directed simulation. The approach of Dwyer and Nachmanson [23] routes on a Yao-graph spanner with local shortcutting. For orthogonal layouts, the libavoid framework of Wybrow, Marriott and Stuckey performs obstacle-avoiding connector routing with incremental updates [43]. Our corridor method eliminates the spanner entirely and routes directly on the CDT dual graph, and is, to our knowledge, the first routing algorithm integrated with a browser-side tile pyramid that reroutes edges per level.

3 System Architecture

MSAGLJS is organized as a monorepo with five NPM packages:

- `@msagl/core` — Graph data structures, layout algorithms (Sugiyama, MDS, IPsep-CoLa), edge routing, CDT, tiling.
- `@msagl/drawing` — Visual attributes (colors, shapes, labels) bridging the graph model and renderers.
- `@msagl/parser` — Parsers for DOT and JSON graph formats.
- `@msagl/rendererer-svg` — SVG-based renderer for small graphs.
- `@msagl/rendererer-webgl` — WebGL renderer using deck.gl [4], with tiling support.

The layout pipeline proceeds in three phases:

1. **Layout.** Node positions are computed by the chosen algorithm. For large undirected graphs the default is force-directed layout, IPsepCola, initialized with Pivot MDS [16]; for directed graphs, Sugiyama layout is used.
2. **Edge routing.** Edges are routed around node obstacles. The routing mode is configurable; this paper focuses on *corridor routing*, Section 5.
3. **Tiling.** A tile hierarchy is built for level-of-detail rendering, Section 4.

103 The WebGL renderer of Section 7 consumes the tile hierarchy and renders the appropriate
 104 level of detail based on the current viewport.

105 4 Tiling

106 To visualize large graphs interactively, MSAGLJS builds a pyramid of tiles analogous to those
 107 used in online maps, served to the browser through the `TileLayer` of the `@deck.gl/geo-layers`
 108 package [12]. Levels are numbered $0, 1, \dots, Z$ from coarsest to finest; level 0 is a single tile
 109 covering the whole drawing with only a few top-ranking graph features, and the finest level Z
 110 corresponds to the full graph produced by the layout and the corridor router of Section 5.

111 Conceptually, each tile is associated with a rectangular viewport and holds the graph
 112 elements that lie within it: the single tile at level 0 is associated with a viewport covering the
 113 entire drawing, so it contains the whole graph. When rendering, every level shows a subset
 114 of a rank-ordered prefix of the graph’s nodes: candidates are processed in order of decreasing
 115 PageRank, and one is accepted only if it can be drawn at its natural size or larger without
 116 its padded bounding box overlapping any already-accepted node; otherwise it is skipped.
 117 Edges are inherited: an edge is shown only when both endpoints are accepted. Each tile then
 118 renders the part of the accepted set that lies within its viewport; the remaining elements
 119 are hidden. The per-tile capacity $\mathcal{C}$ only governs the choice of Z , that is, when to stop
 120 subdividing, and is not consulted when deciding what to render at each level: the selection
 121 of visible elements per level is driven entirely by importance and geometric non-overlap,
 122 Section 4.1. Only the finest level Z displays every node and edge of the graph.

123 *Per-tile rendering load.* At the finest level, condition (i) is intended to cap each tile at
 124 $\mathcal{C}$ elements, nodes plus edge clips, with default $\mathcal{C} = 500$. On dense graphs, however, either
 125 the depth cap $Z_{\max} = 8$ or the minimum tile size (ii) binds first, in which case the densest
 126 finest-level tile may hold many more than $\mathcal{C}$ elements. For a coarser level $z < Z$ we ignore
 127 $\mathcal{C}$ when preparing a tile for rendering. We apply semantic zoom on the nodes, as detailed
 128 in Section 4.1, by inflating each surviving node by roughly 2^{Z-z} in linear size. The chosen
 129 nodes define the rendered edges: we render the edges connecting two chosen nodes. Table 1
 130 reports the empirical worst-case per-tile element count on a range of graphs.

131 4.0.0.1 How Z is chosen.

132 Z is determined incrementally rather than fixed in advance. We initialize $Z = 0$ with the single
 133 root tile covering the whole drawing. While the tiles at level Z violate conditions (i) or (ii)
 134 below, we increase Z by one and split every tile of the previous finest level into four equal
 135 sub-tiles. The two conditions are: (i) every tile contains at most $\mathcal{C}$ elements, with default
 136 $\mathcal{C} = 500$, preventing overloaded tiles; and (ii) the tile width and height are both below ten
 137 times the average node width and height, preventing tiles smaller than the nodes they display.
 138 We additionally enforce a hard cap $Z \leq Z_{\max}$ derived from a memory estimate. At level z
 139 the pyramid has a $2^z \times 2^z$ grid of tiles, so the finest grid resolution is $K = 2^Z$. Across the
 140 whole pyramid we estimate the total number of stored tile elements as

$$141 \quad T(K) \approx 2 \left((K/4) M + N \right),$$

142 where N and M are the node and edge counts of G . The term $(K/4) M$ counts per-tile edge
 143 clips at the finest level, under the heuristic that an average edge crosses $K/4$ tiles. The
 144 additive N counts node copies. The factor 2 accounts for the contribution of all coarser
 145 levels combined. With roughly 200 bytes per stored element and a maximum memory $\mathcal{M}_{\max}$

159 **Algorithm 1** BUILD_TILE_PYRAMID($G, \mathcal{C}, \text{minTileSize}$)

```

160 1:  $T_0 \leftarrow$  single tile holding all of  $G$  clipped to its bounding box
161 2:  $\text{levels} \leftarrow [T_0]$ ;  $z \leftarrow 0$ 
162 3: while some tile in  $\text{levels}[z]$  exceeds  $\mathcal{C}$  elements and its width/height  $\geq \text{minTileSize}$  do
163 4:    $\text{levels}[z+1] \leftarrow$  split every tile of  $\text{levels}[z]$  into four sub-tiles
164 5:   within each parent, distribute nodes/labels/arrowheads by bbox, and split each edge
165   clip at the tile midlines (Section 4.3)
166 6:    $z \leftarrow z + 1$ 
167 7: end while
168 8:  $Z \leftarrow z$ ;  $V_Z \leftarrow V(G)$ ;  $s_v^{(Z)} \leftarrow 1$  for all  $v \in V_Z$ 
169 9: compute PageRank( $G$ )
170 10: for  $z = Z - 1$  down to 0 do
171 11:    $(V_z, \{s_v^{(z)}\}) \leftarrow \text{SELECT\_TOP\_K\_WITH\_ADAPTIVE\_SCALE}(V_{z+1}, 2^{Z-z})$   $\triangleright$  Section 4.1
172 12:   build a CDT from the inflated polylines of  $V_z$  at scales  $\{s_v^{(z)}\}$ 
173 13:   reroute every edge  $e \in E_z$  on this CDT, using the level- $(z+1)$  route as a sleeve hint  $\triangleright$ 
174   Section 4.2
175 14:   for every tile  $t$  at level  $z$ , recompute the clips of  $e$  inside  $t$  from the new curve, splitting
176   at tile midlines
177 15: end for
178 16: return  $\text{levels}$ 

```

146 (default 4 GB), we solve $200T(K) \leq \mathcal{M}_{\max}$ for the largest admissible K ,

$$147 \quad K_{\max} = \left\lfloor 4(\mathcal{M}_{\max}/(2 \cdot 200) - N)/M \right\rfloor,$$

148 and set

$$149 \quad Z_{\max} = \min(8, \lfloor \log_2 K_{\max} \rfloor).$$

150 The constant 8 is a safety ceiling: even when $\mathcal{M}_{\max}$ allows a much larger grid, we never
 151 build more than nine pyramid levels.

152 4.0.0.2 Three phases.

153 Once Z and the tile rectangles are fixed, construction proceeds in three phases for every
 154 coarser level $z = Z-1, Z-2, \dots, 0$: (1) use PageRank to pick the nodes shown at level z and
 155 possibly enlarge them, Section 4.1; (2) reroute the edges of G_z around these obstacles, which
 156 may now be enlarged, Section 4.2; (3) recompute the per-tile edge clips at level z by splitting
 157 the new curves at tile boundaries, Section 4.3. Figure 1 on the first page shows two adjacent
 158 pyramid levels built by this procedure. Algorithm 1 summarizes the whole pipeline.

179 4.1 PageRank-Guided Level Construction

180 To enable semantic zoom and keep the labels readable at coarse zooms, each level $0 \leq z < Z$
 181 is assigned its own *level graph* $G_z = (V_z, E_z)$: a node subset $V_z \subseteq V$, together with every
 182 edge of G whose endpoints both lie in V_z . We compute PageRank [39] on G once, sort all
 183 nodes by descending PageRank as $v_0, v_1, \dots, v_{|V|-1}$. We write $S_z = 2^{Z-z}$ for the per-level
 184 scale *ceiling*.

185 **4.1.0.1 Building V_z .**186 The candidate set at level z is the rank-prefix

187
$$T_z = \{v_0, v_1, \dots, v_{n_z-1}\}, \quad n_z = \lceil |V|/2^{Z-z} \rceil,$$

188 a fixed-size prefix of the globally rank-sorted node list, halving with each coarser level. We
 189 process T_z in rank-descending order. Each candidate v is assigned the largest scale $s_v \leq S_z$ at
 190 which its padded bounding box, scaled about v 's center, stays disjoint from the padded boxes
 191 of all already-accepted nodes; the padding absorbs the corridor router's obstacle inflation and
 192 leaves a visible gap after triangulation. Disjointness is an axis-aligned separating-axis test,
 193 so the largest admissible scale $s^*(v)$ is obtained in closed form. If $s^*(v) \geq 1$ the candidate
 194 is accepted at $s_v = \min(\sigma, s^*(v))$, where the running cap σ , initially S_z , is then tightened
 195 to s_v so that no later, lower-ranked node is drawn larger than an already-accepted senior;
 196 otherwise the candidate is dropped. The accepted set is V_z and the per-node scales $\{s_v^{(z)}\}$ are
 197 stored. The result is a map-like effect: a few large landmarks at coarse zooms, progressively
 198 more detail as the user zooms in.

199 To avoid testing every candidate against every previously accepted box, accepted boxes
 200 are maintained in a uniform spatial hash whose cell size is an upper bound on the inflated
 201 box dimension at the current level. Two boxes can overlap only if their center cells are within
 202 one step on each axis, so each candidate consults at most nine cells. On the layouts we tested
 203 the expected work per candidate is $O(1)$, giving an overall expected cost of $O(|T_z| \log |T_z|)$
 204 dominated by the initial sort; the worst case, when many boxes pile into a single cell, degrades
 205 to $O(|T_z| \cdot |V_z|)$. In our experiments this phase was never a bottleneck: the per-level rerouting
 206 of Section 4.2 dominates the build time at every level we measured.

207 **4.2 Stable Per-Level Rerouting with a Routing Hint**

208 At each level we change the graph and the node sizes, so the edges must be rerouted on each
 209 level as well. We rerun the corridor router, Section 5, on the set of the level edges.

210 **4.2.0.1 Per-level CDT.**

211 For level $z-1$ we build a dedicated CDT from the inflated polylines of V_{z-1} only, using
 212 each node's level- $z-1$ scale. The global landmark v_0 , for example, enters the triangulation as
 213 an obstacle that is 2^{Z-z+1} times its finest-level size in each dimension, so routes naturally bend
 214 around it.

215 **4.2.0.2 Routing hint.**

216 For every edge $e \in E_{z-1}$ we already have a route c_e computed at the finer level z . Sampling c_e
 217 and listing the level- $z-1$ CDT triangles it visits, expanded by one neighbor hop, yields a
 218 triangle *sleeve* S_e that we can use as a per-edge hint for an A* search restricted to S_e . This
 219 keeps the coarse route close to the fine one, so an edge deforms smoothly as the user zooms
 220 out instead of flipping to the other side of a landmark. The price is one independent A* per
 221 edge.

222 By default we instead reuse the per-source Dijkstra trees of Section 5.2, which are faster
 223 and behave well in practice. The reason this option is not always available is that, in a
 224 subgraph, the set of transparent obstacles is per-edge rather than per-source: the boundaries
 225 of nodes on the ancestor chains of the edge's endpoints may be crossed, and for edges sharing
 226 a source these chains differ with the target. A single Dijkstra tree from the source therefore

cannot serve all of its outgoing edges in this setting, so the hint+A* variant is used in this case. The graphs listed in the paper do not have subgraphs.

4.3 Splitting Edges between Tiles

Once each level G_z has its final geometry, we cut that level into tiles. A tile is a pair $(rect, tiledata)$, where *tiledata* is a set of *tile elements*: nodes, edge labels, edge arrowheads, and *edge clips*. An edge clip is a pair (e, p) , with p a continuous piece of the edge curve c_e confined to *rect*.

The single tile at level 0 is obtained by clipping every element to the graph's bounding box. To build level $z + 1$ from level z , each tile is split into four equal sub-tiles; nodes, labels, and arrowheads are assigned to sub-tiles by bounding-box intersection, and each edge clip is cut at the tile midlines, producing sub-clips confined to individual sub-tiles. We maintain invariant $\mathcal{F}$: (a) every clip stays inside its tile's rectangle, crossing the boundary only at its endpoints; (b) the union of all clips for edge e inside a tile equals $c_e \cap rect$. Invariant $\mathcal{F}$ serves two purposes. Part (a) makes subdivision cheap and local. Suppose a parent tile has rectangle *rect* with horizontal and vertical midlines ℓ_h, ℓ_v . To split a clip (e, p) into its children, we collect all intersections of the underlying curve c_e with ℓ_h and ℓ_v , restrict them to the parameter range $[start, end]$ defining p , sort these parameters, and prepend *start* and append *end*. This yields a sorted parameter list $start = t_0 < t_1 < \dots < t_m = end$, and the m child sub-clips are exactly the curve pieces over consecutive intervals $[t_u, t_{u+1}]$. Each piece is assigned to one of the four children by sampling c_e at the midpoint $(t_u + t_{u+1})/2$ and comparing against the midlines. Crucially, no intersection with the four outer sides of *rect* is ever computed: invariant (a) guarantees that the curve already lies inside *rect* between t_0 and t_m and crosses its boundary only at those endpoints, so the midlines alone fully decompose the clip into rectangle-confined pieces. This reduces subdivision to one curve–line intersection per midline plus a sort, instead of a full 2D rectangle clip. Part (b) is the rendering-correctness condition: at any zoom only the tiles intersecting the viewport are fetched and their clips drawn, and (b) guarantees that the union of what is drawn equals $c_e \cap \text{viewport}$ for every visible edge e , so no segment is missed and none is drawn twice. The same invariant is also what makes the rerouting of Section 4.2 compatible with tiling: whenever an edge curve c_e is replaced by a coarser-level route, (b) is reestablished by recomputing the clips of that edge inside every affected tile before the level is served.

Subdivision of a level stops as soon as either of the following holds: (i) every tile at that level already contains at most $\mathcal{C}$ elements; or (ii) the tile width and height have both dropped below ten times the average node width and height, respectively. These are the same conditions that determine Z (Algorithm 1).

Table 1 reports the resulting maximum number of elements rendered in a single tile across the entire pyramid for a range of graphs.

5 Corridor Routing on the CDT

We describe an edge routing method that routes edges directly on the dual graph of a Constrained Delaunay Triangulation, without requiring a spanner graph. The method finds a *corridor*—a sequence of adjacent CDT triangles from source to target—and applies the funnel algorithm to produce the shortest path within it.

■ **Table 1** Maximum number of elements rendered in a single tile across the whole pyramid for several graphs (capacity $\mathcal{C} = 500$, $Z_{\max} = 8$). The capacity $\mathcal{C}$ only guides the choice of Z ; we have no tight analytical bound on the actual rendered per-tile element count, which is therefore reported empirically. “Max tile’s level” is the level on which the maximum tile was found. In the worst cases (**facebook_combined**, **ca-CondMat**) the densest tile holds 20–30 $\times$ the nominal capacity $\mathcal{C}$.

Graph	$ V $	$ E $	Levels built	Max per tile	Max tile’s level
gameofthrones	407	2 639	5	1 168	4
composers	3 405	13 832	8	3 317	7
ca-GrQc	5 241	14 484	8	934	7
ca-HepTh	9 875	25 973	9	1 598	8
facebook_combined	4 039	88 234	9	11 874	8
ca-CondMat	23 133	186 878	9	13 744	8
deezer_europe	28 281	92 752	9	8 201	8
delaunay_n15	32 768	98 274	9	611	8

5.1 CDT Construction and the Dual Graph

Given the graph layout, each node is covered by a padded obstacle polygon. We compute a Constrained Delaunay Triangulation $\mathcal{T}$ on the set $\mathcal{O}$ of these obstacle polygons [18, 20]. The *dual graph* D of $\mathcal{T}$ has one node per triangle and one edge for each pair of triangles sharing a side. A triangle is *free-space* if it is not contained in any obstacle interior.

5.2 Corridor Routing from a Sleeve

Intuitively, each triangle is a short stretch of corridor and shared edges act as “doorways” between consecutive stretches. Routing an edge from s to t splits into two subproblems: a *combinatorial* one—which sequence of doorways to pass through—and a *geometric* one—which exact polyline through that sequence is shortest. The geometric step is the classical funnel algorithm [17, 30], which pulls the path taut inside of the polygon formed by the corridor triangles, the sleeve. The corridor router itself is therefore agnostic to how the sleeve is produced; any search on the dual graph that returns such a sequence will do.

To obtain the sleeve we run a shortest-path search on D where the step from one triangle to a neighbor costs the Euclidean distance between their centroids; triangles inside obstacles other than the source and target obstacles are excluded. The natural per-edge instantiation is A^* [29] with the straight-line distance to t as heuristic. When a node has many incident edges, however, running independent searches is wasteful. We instead group edges by source s and run a single Dijkstra computation per source, expanding until all targets are reached and recovering each sleeve by following parent pointers. This reduces the number of searches from m to at most n , the number of nodes; on the Game of Thrones social network (407 nodes, 2 639 edges, 331 distinct sources) it cuts routing time by 30%. Either variant feeds the same corridor router downstream.

5.3 Vertex Collapse at Source and Target

The corridor passes through triangles whose vertices lie on the padded obstacle boundaries. When these vertices appear as diagonal endpoints, the funnel algorithm might produce sharp turns near each endpoint. We eliminate these artifacts by *collapsing* obstacle vertices: any diagonal endpoint belonging to the source obstacle is replaced with the source center s , and

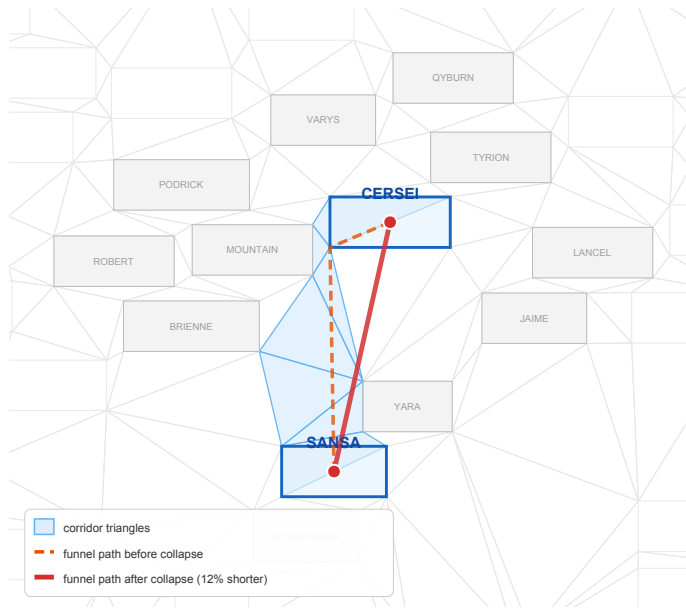


Figure 2 Effect of vertex collapse on the CERSEI–SANSA edge. Light blue: corridor triangles. Orange dashed: funnel path before collapse (3 waypoints). Red solid: funnel path after collapse (2 waypoints, 12% shorter). Collapsing obstacle vertices to the node center widens the funnel opening and removes the detour.

similarly for the target. Degenerate diagonals, those whose endpoints both collapse to the same point, are removed. The left/right orientation of each diagonal is preserved from the uncollapsed geometry.

After collapse, the funnel sees a wide opening at each endpoint and naturally selects the optimal exit direction. A final arrowhead-trimming step clips the path at the node boundary curves.

6 The Portal Graph

The batched Dijkstra approach of Section 5.2 still requires exploring much of the CDT for each source node. To enable faster queries, we introduce the *portal graph*—an abstraction that separates the free-space routing problem from the local obstacle traversal.

6.1 Portal Triangles

For each node obstacle, we identify its *portal triangles*: the free-space CDT triangles that share an edge with the obstacle boundary. Formally, a triangle t is a portal of obstacle O if t is free-space and at least two of its three CDT sites belong to O 's boundary polyline. Portals serve as entry and exit points: any shortest path from the interior of O to free space must pass through a portal triangle.

6.2 Structure of the Portal Graph

The portal graph G_P is defined as follows:

- **Nodes.** For each graph node v , the center point c_v and all portal triangles $\{p_1^v, p_2^v, \dots\}$ of v 's obstacle are nodes of G_P .

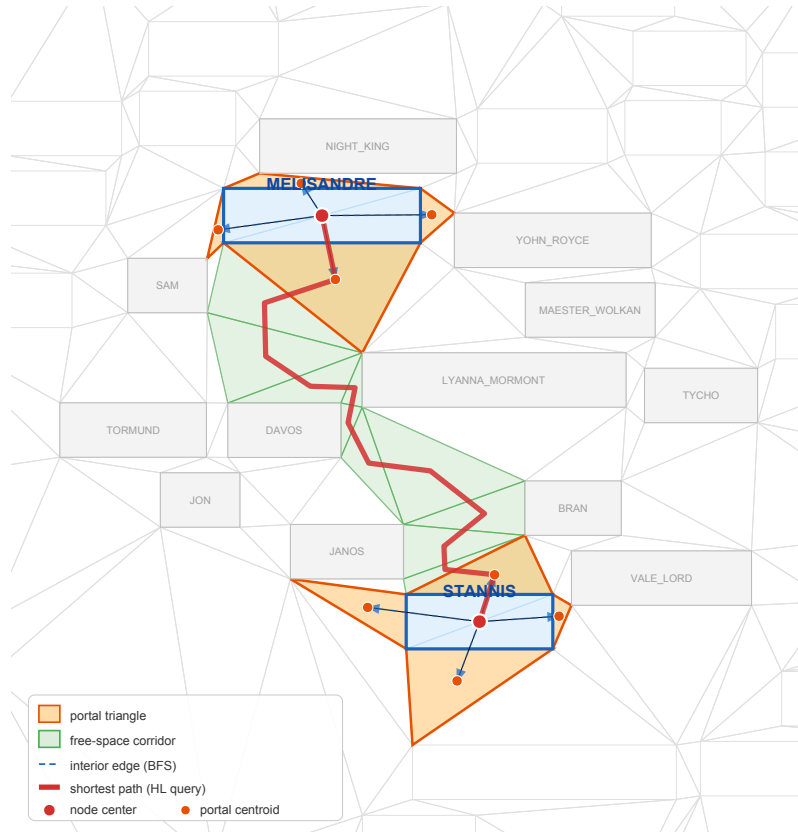


Figure 3 The portal graph for the MELISANDRE-STANNIS edge on the Game of Thrones social network. Orange triangles are portals—free-space CDT triangles adjacent to the obstacle boundary. Blue dashed arrows show interior edges from node centers to portal centroids. Green-shaded triangles form the free-space corridor found by the batched Dijkstra search. The thick red polyline shows the shortest path $c_{\text{MEL}} \rightarrow p_i \rightarrow \dots \rightarrow p_j \rightarrow c_{\text{STAN}}$. Nearby node obstacles are shown in gray.

- **Interior edges.** Each center c_v is connected to every portal p_i^v of the same obstacle, with weight equal to the BFS distance through the obstacle’s interior triangles.
- **Free-space edges.** Portal triangles of different obstacles are connected through the free-space CDT dual graph.

To route a graph edge from node w to node u , we find the shortest path in G_P from c_w to c_u . This path decomposes into three segments:

1. An *interior path* from c_w through w ’s obstacle to some portal p_i^w .
2. A *free-space path* from p_i^w to some portal p_j^u of u .
3. An *interior path* from p_j^u through u ’s obstacle to c_u .

The free-space segment (2) is the bottleneck: it requires a shortest-path query on the free-space dual graph, which may contain thousands of triangles. We answer these queries with the batched Dijkstra trees of Section 5.2.

7 WebGL Rendering

The WebGL renderer uses deck.gl [4], a GPU-accelerated framework for large-scale data visualization. The renderer sets up an `OrthographicView`, with no perspective distortion,

Graph	Nodes	Edges	CDT Δ	Free Δ	Portals	Dijkstra (ms)
GoT [15]	407	2 639	3 258	2 444	1 628	277
b103 [2]	944	2 438	7 554	5 666	3 776	754
b100 [1]	1 463	5 806	11 706	8 780	5 852	2 120
composers [11]	3 405	13 832	27 242	20 432	13 620	9 335
Gnutella04 [7]	10 876	39 994	87 010	65 258	43 504	93 760

Table 2 Corridor routing benchmarks. Dijkstra: per-source Dijkstra trees on the CDT dual.

and a `TileLayer` that dynamically loads tile data based on the current viewport.

7.0.0.1 Tile resolution mapping.

The root tile size is rounded up to the nearest power of two. A *start zoom* level is computed so that the root tile maps to the standard 512-pixel tile size of deck.gl. The `TileLayer`'s `getTileData` callback translates deck.gl tile coordinates (x, y, z) to MSAGLJS tile map coordinates, fetching the precomputed tile data.

7.0.0.2 Rendering layers.

Each tile is rendered as a composite deck.gl layer containing:

- A **GeometryLayer** for node shapes (rectangles, ovals, diamonds).
- A **TextLayer** for node labels.
- A custom **CurveLayer** for edge paths, supporting cubic Bézier curves, arcs, and line segments via GPU-side interpolation. Curve tessellation resolution scales with zoom: at zoom level z , the resolution is 2^{z-2} segments per curve span.
- An **IconLayer** for edge arrowheads.
- A **TextLayer** for edge labels.

7.0.0.3 Style-based filtering.

A declarative style specification allows per-layer control of visibility ranges (`minZoom`, `maxZoom`), colors, opacity, sizes, and filters. Styles can reference the node's PageRank to, e.g., show labels only for important nodes at medium zoom levels.

7.0.0.4 Interaction.

The renderer supports pan, zoom, hover highlighting, and click selection. Hovering an edge highlights it and its incident nodes. The `zoomTo` API animates the viewport to a target rectangle with smooth transitions.

8 Experimental Results

We evaluate the corridor routing method on benchmark graphs from the Game of Thrones social network [15], the GD 2011 contest [11], the Graphviz test suite [2, 1], and the Gnutella P2P network [7]. All experiments run in Node.js on a laptop with an Apple M1 processor. Layout uses Pivot MDS; timings include CDT construction, routing, and curve smoothing.

Table 2 reports the performance of the Dijkstra-tree corridor router, Section 5.2. For each graph, we report the number of CDT triangles, free-space triangles, the total number of portal triangles across all nodes, and the total routing time.

The Dijkstra-tree approach scales to graphs with tens of thousands of edges. Routing the full Gnutella04 graph (10 876 nodes, 39 994 edges) on an Apple M1 laptop takes about 94 s,

which is amortized in interactive use because tile pyramids cache the routes computed at the finest level and rerun only the per-level A* described in Section 4 on coarser zooms.

9 Conclusion

We presented MSAGLJS, an open-source TypeScript library for graph layout, edge routing, and visualization in web browsers. The corridor routing method routes edges directly on the CDT dual graph using a portal graph abstraction and the funnel algorithm. The tiling scheme enables map-like exploration of large graphs with level-of-detail filtering and edge simplification. The WebGL renderer, built on deck.gl, provides smooth pan-and-zoom interaction.

MSAGLJS is available at <https://github.com/microsoft/msagljs> under the MIT license.

References

- 1 b100. <https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b100.dot>.
- 2 b103. <https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b103.dot>.
- 3 Cosmograph. <https://cosmograph.app>.
- 4 deck.gl: Large-scale WebGL-powered data visualization. <https://deck.gl/>.
- 5 Funnel algorithm. <https://page.mi.fu-berlin.de/mulzer/notes/alggeo/polySP.pdf>.
- 6 Graphviz. <http://www.graphviz.org/>.
- 7 p2p-gnutella04. <https://snap.stanford.edu/data/p2p-Gnutella04.html>.
- 8 Regraph. <https://cambridge-intelligence.com/regraph/>.
- 9 sdfp. <https://graphviz.org/docs/layouts/sfdp/>.
- 10 Sigma.js: a JavaScript library aimed at visualizing graphs of thousands of nodes and edges. <https://www.sigmajavascript.org/>.
- 11 Skewed. <http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml>.
- 12 TileLayer (@deck.gl/geo-layers). <https://deck.gl/docs/api-reference/geo-layers/tile-layer>. Accessed: 2026.
- 13 yworks. <https://yworks.com/products/yed>.
- 14 James Abello, Frank Van Ham, and Neeraj Krishnan. ASK-GraphView: A large scale graph visualization system. In *IEEE Transactions on Visualization and Computer Graphics*, volume 12, pages 669–676. IEEE, 2006.
- 15 Andrew Beveridge and Michael Chemers. The game of game of thrones: Networked concordances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pages 201–225. Routledge, 2018.
- 16 Ulrik Brandes and Christian Pich. Eigensolver methods for progressive multidimensional scaling of large data. In *Graph Drawing: 14th International Symposium, GD 2006, Karlsruhe, Germany, September 18–20, 2006. Revised Papers 14*, pages 42–53. Springer, 2007.
- 17 Bernard Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 339–349. IEEE, 1982.
- 18 Boris Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i Estestvennyka Nauk*, 7(1):793–800, 1934.
- 19 David P Dobkin, Emden R Gansner, Eleftherios Koutsofios, and Stephen C North. Implementing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD'97 Rome, Italy, September 18–20, 1997 Proceedings 5*, pages 262–271. Springer, 1997.
- 20 Vid Domiter and Borut Žalik. Sweep-line algorithm for constrained delaunay triangulation. *International Journal of Geographical Information Science*, 22(4):449–462, 2008.

- 442 21 Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier
443 for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium*
444 *on Theory of Computing (STOC)*, 2025. <https://arxiv.org/abs/2504.17033>.
- 445 22 Tim Dwyer, Yehuda Koren, and Kim Marriott. IPSep-CoLa: An incremental procedure for
446 separation constraint layout of graphs. *IEEE Transactions on Visualization and Computer*
447 *Graphics*, 12(5):821–828, 2006. doi:10.1109/TVCG.2006.156.
- 448 23 Tim Dwyer and Lev Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing:*
449 *17th International Symposium, GD 2009, Chicago, IL, USA, September 22-25, 2009. Revised*
450 *Papers 17*, pages 147–158. Springer, 2010.
- 451 24 Max Franz, Christian T. Lopes, Gerardo Huck, Yue Dong, Onur Sumer, and Gary D. Bader.
452 Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2):309–
453 311, 2016.
- 454 25 Emden R. Gansner, Yehuda Koren, and Stephen North. Topological fisheye views for visualizing
455 large graphs. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):457–468,
456 2005.
- 457 26 Emden R. Gansner and Stephen C. North. An open graph visualization system and its
458 applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233,
459 2000.
- 460 27 Leonidas Guibas, John Hershberger, Daniel Leven, Micha Sharir, and Robert E. Tarjan.
461 Linear-time algorithms for visibility and shortest path problems inside triangulated simple
462 polygons. *Algorithmica*, 2(1–4):209–233, 1987.
- 463 28 David Harel and Yehuda Koren. A fast multi-scale method for drawing large graphs. In
464 *Journal of Graph Algorithms and Applications*, volume 6, pages 179–202, 2002.
- 465 29 Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic
466 determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*,
467 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136.
- 468 30 John Hershberger and Jack Snoeyink. Computing minimum length paths of a given homotopy
469 class. *Computational geometry*, 4(2):63–97, 1994.
- 470 31 John Hershberger and Subhash Suri. An optimal algorithm for Euclidean shortest paths in
471 the plane. *SIAM Journal on Computing*, 28(6):2215–2256, 1999.
- 472 32 Danny Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical
473 data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006.
- 474 33 Danny Holten and Jarke J. Van Wijk. Force-directed edge bundling for graph visualization.
475 In *Computer Graphics Forum*, volume 28, pages 983–990. Wiley Online Library, 2009.
- 476 34 Yifan Hu. Efficient, high-quality force-directed graph drawing. *Mathematica Journal*, 10(1):37–
477 71, 2005.
- 478 35 Christophe Hurter, Ozan Ersoy, and Alexandru Telea. Graph bundling by kernel density
479 estimation. In *Computer graphics forum*, volume 31, pages 865–874. Wiley Online Library,
480 2012.
- 481 36 D. T. Lee and Franco P. Preparata. Euclidean shortest paths in the presence of rectilinear
482 barriers. *Networks*, 14(3):393–410, 1984.
- 483 37 Lev Nachmanson, Roman Prutkin, Bongshin Lee, Nathalie Henry Riche, Alexander E Holroyd,
484 and Xiaoji Chen. Graphmaps: Browsing large graphs as interactive maps. In *Graph Drawing*
485 *and Network Visualization: 23rd International Symposium, GD 2015, Los Angeles, CA, USA,*
486 *September 24-26, 2015, Revised Selected Papers 23*, pages 3–15. Springer, 2015.
- 487 38 Lev Nachmanson, George G. Robertson, and Bongshin Lee. Drawing graphs with GLEE. In
488 *Graph Drawing: 15th International Symposium, GD 2007*, pages 389–394. Springer, 2008.
- 489 39 Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation
490 ranking: Bringing order to the web. *Stanford InfoLab*, 249(373):1–17, 1999.
- 491 40 Alexandre Perrot and David Auber. Cornac: Tackling huge graph visualization with big data
492 infrastructure. *IEEE Transactions on Big Data*, 6(1):80–92, 2018.

- 493 **41** Jonathan Richard Shewchuk. Delaunay refinement algorithms for triangular mesh generation.
494 *Computational Geometry*, 22(1–3):21–74, 2002.
- 495 **42** Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiro Toda. Methods for visual understanding
496 of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*,
497 11(2):109–125, 1981. doi:10.1109/TSMC.1981.4308636.
- 498 **43** Michael Wybrow, Kim Marriott, and Peter J. Stuckey. Orthogonal connector routing. *Lecture*
499 *Notes in Computer Science*, 5849:219–231, 2010.